

Classical String Matching Meets Soft Computing: A Controlled Empirical Study of Hybrid Intrusion Detection

L Dileep Sai

Dept of CSE,

SRM University AP

Amaravati, Andhra Pradesh

dileep_lavu@srmap.edu.in

K Sagar

Dept of CSE,

SRM University AP

Amaravati, Andhra Pradesh

sagar_kota@srmap.edu.in

P Naga Venkata Sai Shankar

Dept of CSE,

SRM University AP

Amaravati, Andhra Pradesh

shankar_patnala@srmap.edu.in

M Sitharam

Dept of CSE,

SRM University AP

Amaravati, Andhra Pradesh

sitharam.m@srmap.edu.in

Abstract—Signature-based intrusion detection systems (IDS) find attacks by searching network traffic for known patterns, so their speed depends on the string-matching algorithm they use. These algorithms are usually compared only by their theoretical complexity, and the number of attacks that exact matching misses is rarely measured. We compare four algorithms, Knuth–Morris–Pratt (KMP), Rabin–Karp, Aho–Corasick and a Bloom filter, on KDD Cup 99, NSL-KDD and CICIDS2017. As these datasets have no payloads, each record is turned into a short text of its traffic features, without the class label, and signatures are learned from training data only. All four algorithms detect the same attacks, but their speeds differ by over 100 times: with 10,000 signatures, Aho–Corasick still handles over 50,000 records per second, while KMP and Rabin–Karp slow down as signatures are added. The usual Bloom-filter false-positive formula holds only with independent hash functions. Checking records with signatures first, and sending only unmatched records to a machine learning classifier, improves F1 over each of three classifiers used alone in all ten runs on every dataset (up to 0.987, 0.825 and 0.913). However, completely new attack families are still mostly missed, although new variants of known attacks are often caught. Further tests on data leakage, tuning, hidden attack families and padding attacks support these results.

Index Terms—Intrusion detection system, string matching, hybrid intelligent systems, KDD Cup 99, NSL-KDD, CICIDS2017.

I. INTRODUCTION

Most intrusion detection systems (IDS) in use today, such as Snort [1], [2], find attacks by searching network traffic for patterns of known attacks. This search can take up to 70% of Snort’s running time [3], so the string-matching algorithm limits how much traffic an IDS can check. Classical algorithms, namely Knuth–Morris–Pratt (KMP) [4], Rabin–Karp [5], Aho–Corasick [6] and Bloom filters [7], are well understood in theory, and this theory is often the only reason given for choosing one of them. Exact matching also only finds attacks that its signature list already describes, a limit that soft-computing methods try to overcome.

These two limits raise two questions. First, do the theoretical advantages of these algorithms, and the textbook false-positive formula of Bloom filters, still hold in practice when the signature list grows? Second, how many of the attacks missed

by exact matching can a machine learning classifier recover? A fair comparison needs signatures learned from traffic rather than from class labels, and unknown attacks that no trained part of the system has seen.

Systems that combine signatures and machine learning are not new [2], [8], and we do not propose a new design. Instead, we measure, on the same data, matcher speed, Bloom-filter behaviour, and how well a classifier catches attacks that signatures miss. Our contributions are:

- a test setup without label leakage: records turned into label-free text, signatures learned from training data only, and attack types kept out of training as unknown attacks, on KDD Cup 99 [9], NSL-KDD [10] and CICIDS2017 [11];
- speed and memory tests of four matchers with 12 to 10,000 signatures, and Bloom-filter false positives under four hashing methods;
- two-stage detectors with three classifiers over ten runs, with confidence intervals and significance tests;
- robustness tests for duplicates, data split, hidden attack families, tuning and padding attacks, and a C version of the matchers.

We find that measured speed, not complexity, separates the matchers; that the Bloom formula holds only with independent hashing; and that the two stages complement each other but still miss completely new attack families.

II. RELATED WORK

String matching for intrusion detection. Signature-based detection [12] comes down to exact pattern matching. KMP [4] and Rabin–Karp [5] search for one pattern at a time, while Aho–Corasick [6] searches for all patterns in a single pass. Aho–Corasick has become a standard engine in signature-based IDS [1], and memory-saving versions exist for high-speed networks [3].

Bloom filters in packet inspection. A Bloom filter [7] checks whether a string belongs to a set quickly and in little memory, at the cost of occasional false positives. They are widely used in networking [13], including deep packet

inspection [14]. Double hashing computes all k hash values from two hashes; it matches the classical analysis only for very large filters [15], and enhanced double hashing reduces its errors [16].

Soft computing and hybrid IDS. Neural networks were applied to KDD data in 2002 [17], deep models followed [18], and surveys cover machine learning IDS in general [8]. These models can detect unseen attacks, but they suffer when test traffic differs from training traffic [19] and can be fooled by adversarial inputs [20]. KDD Cup 99 also contains many duplicates [10], which is why we add NSL-KDD and CICIDS2017. To our knowledge, no study measures matcher speed, hashing behaviour and classifier fallback on the same records while keeping unknown attacks out of all training.

III. BACKGROUND

Let S be a list of N signature strings and x the text of one record. An exact detector raises an alarm for x if some signature in S appears in x . It never raises an alarm without a matching signature (it is *sound*) and never misses a listed signature (it is *complete*), but it cannot see attacks whose text contains no signature from S . A Bloom filter stores each signature by setting k bits, chosen by hash functions, in an array of M bits; a query is reported as present if all its k bits are set. If the hash functions are independent and uniform, the false-positive probability is

$$p \approx \left(1 - e^{-kN/M}\right)^k. \quad (1)$$

In our matcher, each Bloom-filter hit is checked against the exact signature list, which removes its false alarms. For a record of n characters, KMP and Rabin–Karp scan once per signature ($O(Nn)$), Aho–Corasick scans once ($O(n)$ plus matches), and our Bloom matcher hashes every window for each of the L signature lengths ($O(kLn)$).

IV. METHODOLOGY

A. Pipeline and Threat Model

Fig. 1 shows the two-stage detector. Stage 1 searches the record’s text with one of the four matchers. If no signature matches, stage 2, a machine learning classifier, decides from the record’s features. We consider four kinds of attacker: (A1) one using a registered attack, which stage 1 should detect; (A2) one using an attack with no signature, which only stage 2 can detect; (A3) one who pads traffic to avoid signatures; and (A4) one who changes feature values to fool the classifier. We test A1–A3; A4 is out of scope.

B. Datasets and Pseudo-Packets

None of the three datasets contains packet payloads. *KDD Cup 99*: we use the 10% training set [9], with 494,021 records, 41 features and 23 labels (*normal* or one of 22 attack types). We turn each record into a *pseudo-packet*, a line of 27 words built from observable features (protocol, service, flag, bytes, duration, host indicators and traffic statistics, with counts in logarithmic bins), for example:

```
tcp http SF src=181 dst=5450 dur=b0 ...
cnt=b4 srv=b4 serr=0.00 rerr=0.00 same=1.00
diff=0.00 dhc=b4 dhs=b4
```

The class label is never written into the text. Twelve attack types, which make up 99.9% of attack records (the full list is in the code release), are *registered*; the other ten (*pod*, *land*, *imap*, *rootkit*, *loadmodule*, *ftp_write*, *multihop*, *phf*, *perl*, *spy*) are *unregistered*. The test set contains $\min(300, \lfloor n_c/2 \rfloor)$ records of each registered type c , where n_c is the number of records of type c , plus all 340 unregistered records and 5,000 normal records (7,906 in total). The other 486,115 records form the training set, which has no unregistered attack.

NSL-KDD [10] is KDD Cup 99 without duplicates. We train on KDDTrain+ (125,973 records) and test on KDDTest+ (22,544) and its harder subset KDDTest-21 (11,850). The same twelve types are registered; the ten other KDDTrain+ attack types are seen only by the classifiers, and the 17 types (3,750 records) that appear only in KDDTest+ are unknown.

CICIDS2017 [11] contains 2,830,743 network flows recorded with CICFlowMeter, each with 77 numeric features. We turn each flow into 27 words (port, counts and lengths per direction, TCP flags, window sizes, binned timings). Seven types are registered (DoS Hulk, PortScan, DDoS, DoS GoldenEye, FTP-Patator, SSH-Patator, web brute force). Seven are unregistered (13,981 flows): two slow DoS tools and two web attacks, which are variants of registered families (a family is a group of related attack types, e.g. four DoS tools), and Bot, Infiltration and Heartbleed, which are new families. The test set contains $\min(1,000, \lfloor n_c/2 \rfloor)$ flows of each registered type, all unregistered flows and 20,000 benign flows (40,734 in total).

Terminology. *Registered* attacks have learned signatures. *Unknown* attacks belong to types that never appear in training (unregistered, or novel in NSL-KDD).

Why flow-level signatures? Labelled IDS datasets rarely include payloads, but real rule languages already match flow properties: Snort’s *dsize*, *flags* and *window* options check payload size, TCP flags and window size [1]. Writing these properties as text lets us learn signatures from labelled data. Matcher speed depends only on text length and the signature list, which we also test with payload-sized inputs (Section VI-A).

C. Signature Learning

Signatures are learned from training records only. For each registered type, the candidates are sequences of up to six consecutive words (n -grams) from up to 3,000 sampled pseudo-packets. A candidate is allowed only if it appears in at most $\epsilon = 10^{-4}$ of the normal training records (92,278 records on KDD Cup 99; a sample of 200,000 benign flows on CICIDS2017). A greedy method then repeatedly picks the allowed candidate that covers the most records not yet covered (preferring fewer words), and stops when 99.5% of records are covered, when the gain drops below 0.5%, or after 25 signatures. On KDD Cup 99 this gives 35 signatures of 18 lengths, e.g. `src=1032 dst=0 (smurf)` and `http RSTR`

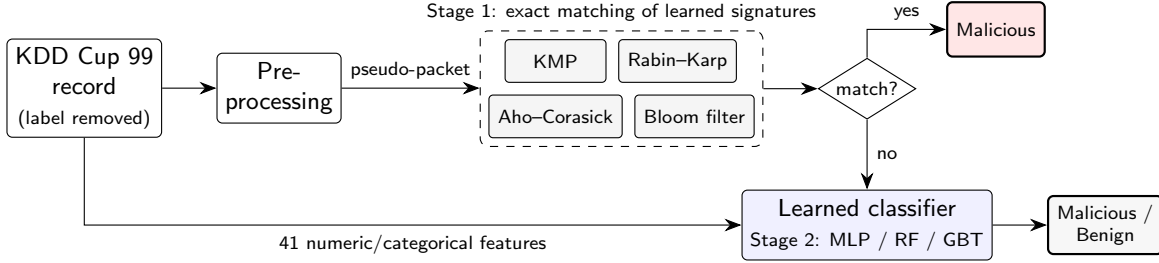


Fig. 1. Two-stage detector: stage 2 runs only when stage 1 finds no learned signature in the record’s label-free text.

(back), covering 73% (*buffer_overflow*, 15 training records) to 100% of each type’s training records.

D. Classifiers and Two-Stage Detection

Stage 2 uses one of three classifiers: a multilayer perceptron (MLP) with hidden layers of 64 and 32 ReLU units, trained with Adam [21] and early stopping; a random forest (RF) of 100 trees; and histogram-based gradient-boosted trees (GBT), all from scikit-learn [22]. They use the 41 KDD features (categorical ones one-hot encoded, 116 inputs) or the 77 CICIDS2017 features (infinite or missing values set to 0), all standardized. Each classifier learns to separate attack from normal records, using at most 20,000 training records per type. The two-stage detector (*cascade*) marks a record as an attack if stage 1 finds a match, and otherwise uses the classifier’s decision.

V. EXPERIMENTAL SETUP

All matchers are written in pure Python 3.11, run on one thread of a Linux container with two Intel Xeon virtual CPUs, and check every signature for every record, as a rule engine must. The four matchers give exactly the same decisions on every test record of all three datasets, and so does a C version (gcc -O2) on the KDD Cup 99 test set. Speed is measured on a separate random sample of 20,000 KDD Cup 99 records. To study how cost grows, we cut the signature list to $N = 12$ or extend it to $N = 10,000$ with extra n -grams from registered training records; Bloom filters use $M = \max(80,000, 20N)$ bits and $k = 7$. The Bloom-filter test checks every window in 3,000 records with four hashing methods and M from 100 to 51,200 bits.

We treat *attack* as the positive class and report F1, the false-positive rate (FPR) and recall per attack group and type. Each classifier is trained with ten random seeds (runs; 42–51), and each seed draws its own training sample. We report means with 95% confidence intervals (t -distribution) and compare detectors with two-sided Wilcoxon signed-rank tests over the paired seeds.

Robustness tests. On CICIDS2017 we (i) remove test flows whose feature values also appear in training, (ii) test on the last flows of each type in file order instead of a random sample, with signatures learned again, and (iii) leave each attack family out of all training in turn (three seeds). On all datasets we (iv)

TABLE I
MATCHER SPEED (EVERY SIGNATURE CHECKED FOR EVERY RECORD) IN PYTHON AND C; THE LAST COLUMN USES 1,500-BYTE TEXTS AND 1,000 SIGNATURES.

Matcher	Python, $N = 35$		C, records/s		
	records/s	Memory (KiB)	$N = 35$	$N = 10^4$	1,500 B
KMP	988	8.1	90,537	333	711
Rabin-Karp	615	5.8	18,383	68	80
Aho-Corasick	79,744	53.7	1,665,418	876,117	188,661
Bloom filter	176	14.4	19,855	1,597	437

tune each classifier on 20% of its training sample (MLP: layer sizes, L2 penalty, log-scaled inputs; RF, GBT: tree settings) and (v) pad every attack record with δ bytes from the attacker’s side, updating its features and text.

Code, signatures and seeds are available from the corresponding author on request.

VI. RESULTS

A. Matcher Speed and Scalability

Aho-Corasick is by far the fastest matcher, and speeds differ by more than 100 times (Table I). With 35 signatures, Aho-Corasick in Python processes 79,744 records per second, 81 times faster than KMP and 130 times faster than Rabin-Karp. The Bloom matcher reaches only 176 records per second, as it hashes every window for each of the 18 signature lengths. As the signature list grows (Fig. 2a,b), KMP and Rabin-Karp slow down in proportion to N , to 3.4 and 2.5 records per second at 10,000 signatures. Aho-Corasick never falls below 50,573 records per second, although its automaton grows to 16.3 MiB. In C (one core), Aho-Corasick is still at least ten times faster than the others: it processes 1.67 million records per second with 35 signatures (18 times KMP), 876,117 with 10,000 signatures (KMP: 333), and 188,661 records per second, about 2.3 Gbit/s and 265 times KMP, on 1,500-byte payload-sized texts with 1,000 signatures.

B. Bloom-Filter False Positives

Over 222,058 distinct windows that are not signatures (Fig. 2c), the false positives with k independent hash functions stay at or near Eq. (1) from $M = 200$ bits upward. Double hashing moves away from the formula as the filter grows: at $M = 1,600$ bits, where 0.26 false positives are expected, it gives 15 with BLAKE2b and 21 with CRC32. CRC32 does

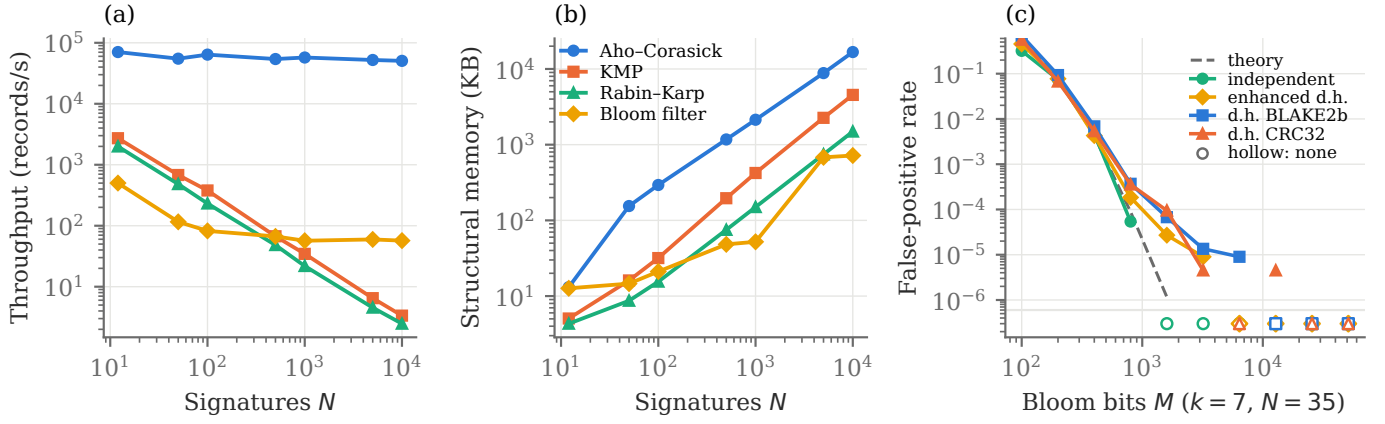


Fig. 2. (a) Speed and (b) memory as the number of signatures grows (Python); (c) Bloom-filter false-positive rate versus filter size (d.h.: double hashing; dashed line: Eq. (1); hollow markers: no false positive).

TABLE II

DETECTION RESULTS ON THE THREE TEST SETS (CLASSIFIERS: MEAN OVER TEN SEEDS). EXACT + X: CASCADE OF EXACT MATCHING AND CLASSIFIER X; REG./UNK.: RECALL ON REGISTERED/UNKNOWN ATTACKS. NSL-KDD: KDDTest+, 95% CI HALF-WIDTHS FOR F1 AND UNK. ARE AT MOST 0.035, EXCEPT UNK. OF TREE-BASED DETECTORS ON KDD CUP 99 (UP TO 0.105) AND MLP-BASED DETECTORS ON CICIDS2017 (F1 0.047, UNK. 0.086).

Detector	KDD Cup 99				NSL-KDD				CICIDS2017			
	F1	FPR%	Reg.	Unk.	F1	FPR%	Reg.	Unk.	F1	FPR%	Reg.	Unk.
Exact	0.937	0.04	0.992	0.044	0.728	2.34	0.731	0.239	0.479	0.21	0.849	0.058
MLP	0.982	0.28	0.989	0.822	0.803	6.25	0.792	0.486	0.691	3.78	0.997	0.335
RF	0.956	0.04	0.997	0.305	0.793	2.84	0.779	0.412	0.779	0.11	0.998	0.466
GBT	0.947	0.06	0.998	0.169	0.813	2.99	0.793	0.481	0.910	0.16	0.999	0.757
Exact + MLP	0.987	0.30	0.999	0.836	0.820	6.44	0.827	0.493	0.703	3.94	0.999	0.357
Exact + RF	0.957	0.08	0.999	0.312	0.808	2.93	0.801	0.435	0.789	0.28	0.999	0.487
Exact + GBT	0.948	0.10	0.999	0.178	0.825	3.06	0.815	0.491	0.913	0.33	1.000	0.767

worse because it is an affine function of its input bits, which makes its two base hashes depend on each other. Enhanced double hashing [16] reduces the gap (6 false positives), as expected from [15]. The 80,000-bit filter used in Table I is safe with all four methods.

C. Detection on Three Datasets

On all three datasets, every cascade beats its classifier in all ten seeds (Wilcoxon $p = 0.002$; Table II). On KDD Cup 99, stage 1 (signatures alone) is precise but narrow: it reaches F1 0.937 with only two false positives (FPR 0.04%) and finds 99.2% of registered attacks, but only 4.4% of unknown attacks. The cascades win because stage 1 catches registered attacks that the classifier misses. The tree models give the fewest false alarms but find few unknown attacks; the MLP finds the most, and its cascade reaches the highest F1, 0.987 ± 0.002 (95% CI). Most unknown attacks are *pod* records (264 of 340), which the MLP finds almost completely; on the other 76 records, the GBT cascade does better (0.63) than the MLP cascade (0.41).

NSL-KDD tests the method when test traffic differs from training traffic, since KDDTest+ was collected separately. KDDTrain+ gives 66 signatures, 19 of them the same as on KDD Cup 99; *warezmaster* (20 training records) gets none. The cascades gain a median of 1.1–1.7 F1 points on KDDTest+ and 1.7–2.5 on KDDTest-21 (all ten seeds, $p = 0.002$).

Signatures help most when there is little training data: for *guess_passwd* (53 training and 1,231 test records), signatures find 15.1% of attacks, the classifiers at most 7.9%, and the MLP cascade 23.0%. On KDDTest+, stage 1 is still the most precise detector (precision 0.971), but its FPR rises from 0.15% on normal training records to 2.34%. The cascades find 43.5–49.3% of unknown attacks. KDD Cup 99 signatures used unchanged reach F1 0.713 (NSL-KDD signatures: 0.728).

CICIDS2017 tests newer traffic and shows where learned signatures reach their limits. The 43 signatures cover over 96% of the training flows for five registered types, but only 67% for PortScan and 20% for web brute force, whose flows look like benign traffic. Since training and test flows come from the same capture, stage 1 keeps its training FPR (0.18% vs. 0.21%). The cascades also raise unknown-attack recall by a median of 1.0–2.2 points. The MLP varies from seed to seed (F1 0.691 ± 0.047), and its 3.8% FPR comes mostly from features with very large values (Section VI-D); GBT finds 75.7% of unknown flows at 0.16% FPR. The GBT cascade finds 93% of *slowloris*, 85% of *Slowhttptest* and 98% of XSS flows, which are variants of registered families, but none of the Bot, Infiltration or Heartbleed flows. No detector’s mean recall is above 3% on Bot or 26% on Infiltration.

TABLE III

ROBUSTNESS TESTS (MEANS OVER TEN SEEDS; THREE FOR THE FAMILY HOLD-OUT). WINS: SEEDS IN WHICH THE CASCADE HAS A HIGHER F1 THAN ITS CLASSIFIER. (B) OMITTS CICIDS2017, WHERE TUNING SELECTS THE DEFAULT GBT.

Test	Model	Exact	Classifier	Cascade	Wins
<i>(a) CICIDS2017 F1 with duplicates removed or split by file order</i>					
Random split (Table II)	GBT	0.479	0.910	0.913	10/10
Duplicates removed	GBT	0.425	0.902	0.905	10/10
Split by file order	GBT	0.426	0.909	0.911	10/10
<i>(b) F1 with tuned classifiers (best one)</i>					
KDD Cup 99	MLP	0.937	0.987	0.990	10/10
NSL-KDD	MLP	0.728	0.807	0.819	10/10
<i>(c) Registered recall with 64 bytes of padding (no padding in brackets)</i>					
KDD Cup 99	MLP	0.422 (0.992)	0.989 (0.989)	0.991 (0.999)	–
NSL-KDD	GBT	0.595 (0.731)	0.710 (0.793)	0.754 (0.815)	–
CICIDS2017	GBT	0.621 (0.849)	0.999 (0.999)	0.999 (1.000)	–

<i>(d) CICIDS2017 recall on a held-out family (best classifier)</i>					
Family (n)	Model	Exact	Classifier	Cascade	
DoS (252,661)	MLP	0.004	0.487	0.488	
DDoS (128,027)	RF	0.001	0.635	0.636	
PortScan (158,930)	MLP	0.002	0.021	0.023	
Brute force (13,835)	MLP	0.016	0.203	0.219	
Web (2,180)	GBT	0.001	0.923	0.923	
Bot (1,966)	MLP	0.000	0.000	0.000	
Infiltration (36)	MLP	0.111	0.194	0.250	
Heartbleed (11)	GBT	1.000	0.667	1.000	

D. Robustness Tests

Duplicates and split. 3,942 CICIDS2017 test flows (9.7%, mostly benign, PortScan and Patator) have an identical copy in the training data. Removing them, or testing on the last flows of each type, lowers the GBT cascade’s F1 only from 0.913 to 0.905 and 0.911, and every cascade still beats its classifier in all ten seeds (Table IIIa).

Tuning. Default settings might favour the cascade. Tuning changes the best cascade F1 by less than one point, and all nine tuned classifiers are still beaten by their cascades in every seed (Table IIIb). On CICIDS2017, log-scaled inputs cut the MLP’s FPR from 3.8% to 0.48%, but its F1 still varies from seed to seed.

Padding attacks (A3). Adding 64 extra bytes (1–512 in the code release) to every attack record breaks all signatures that contain exact byte counts: registered recall of exact matching falls to 0.422, 0.595 and 0.621 on KDD Cup 99, NSL-KDD and CICIDS2017 (Table IIIc). The classifiers are affected differently, and the two stages cover for each other: on KDD Cup 99 the MLP cascade still reaches 0.991, while on CICIDS2017 the signatures lift the RF, which drops to 0.595, back to 0.830.

Hidden attack families. Our choice of unknown types could bias the results. When each CICIDS2017 family is left out of training in turn, the best cascade finds 92% of web attacks, 64% of DDoS and 49% of DoS flows, but at most 22% of brute force, 2% of PortScan and almost no Bot flows; the 11 Heartbleed flows are caught by signatures learned for other types (Table III d). Detection thus depends on how similar a new family is to known ones. In four of these tests the RF and GBT cascades have slightly lower F1 than their classifier, as stage 1 adds false alarms (FPR about 0.2%) but no new detections.

VII. DISCUSSION AND THREATS TO VALIDITY

Our results suggest three practical lessons. First, choose a matcher by measured speed: Aho–Corasick was the fastest in Python and C, while per-signature matchers slow down as the list grows. Second, Bloom pre-filters need independent

hash functions or a large filter. Third, the two stages fail in different ways: stage 1 is precise and catches attacks with little training data, while classifiers keep most of their recall when padding breaks signatures. Neither stage reliably finds new attack families, and a false-alarm limit set on training traffic fails when test traffic differs.

Threats to validity. Our signatures describe flow properties, not payload bytes, and we do not test A4. NSL-KDD shares KDD Cup 99’s 1998 traffic, and CICIDS2017 is synthetic with known labelling errors [23]. Many unknown types have under 40 test records.

VIII. CONCLUSION

This paper asked two questions: does theoretical complexity predict the real speed of string-matching algorithms in an IDS, and how many of the attacks missed by exact matching can a machine learning classifier recover? To answer them fairly, we turned three public datasets (KDD Cup 99, NSL-KDD and CICIDS2017) into label-free text, learned signatures from training data only, kept unknown attack types out of all training, and repeated the main experiments with ten random seeds.

The answer to the first question is no. All four matchers made exactly the same decisions, so the choice between them is purely a question of speed. Aho–Corasick was the fastest in every test, in Python and in C, and stayed above 50,000 records per second in Python with 10,000 signatures, while KMP and Rabin–Karp slowed down as signatures were added. In C, Aho–Corasick processed 1.67 million records per second with 35 signatures, 18 times faster than KMP, and about 2.3 Gbit/s on packet-sized texts with 1,000 signatures. The textbook Bloom-filter formula held only with independent hash functions; double hashing gave far more false positives in small filters, so the formula should not be used to size filters that rely on double hashing.

For the second question, a classifier placed after signature matching improved F1 on all three datasets in every run, reaching 0.987, 0.825 and 0.913. These gains held on a deduplicated test set, when testing on the last flows in file order and with tuned classifiers, and the two stages covered

for each other when padding broke the signatures. However, completely new attack families, such as Bot and Infiltration, were still mostly missed, and when a whole family was held out the cascade sometimes scored slightly below the classifier alone. Neither signatures nor classifiers are therefore enough on their own for truly new attacks.

The practical lesson is simple: use Aho–Corasick for the exact stage, size Bloom filters with independent hashes, and add a learned second stage to catch variants of known attacks, while treating detection of unseen families as an open problem. In future work, we will test attacks that change feature values, use datasets with packet payloads, evaluate the detector on real network traffic, and study faster implementations for high-speed links.

REFERENCES

- [1] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. USENIX LISA*, 1999, pp. 229–238.
- [2] K. Scarfone and P. Mell, “Guide to intrusion detection and prevention systems (IDPS),” NIST, Special Publication 800-94, 2007.
- [3] N. Tuck, T. Sherwood, B. Calder, and G. Varghese, “Deterministic memory-efficient string matching algorithms for intrusion detection,” in *Proc. IEEE INFOCOM*, vol. 4, 2004, pp. 2628–2639, doi: 10.1109/INFCOM.2004.1354682.
- [4] D. E. Knuth, J. H. Morris, and V. R. Pratt, “Fast pattern matching in strings,” *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [5] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” *IBM Journal of Research and Development*, vol. 31, no. 2, pp. 249–260, 1987.
- [6] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [7] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [8] A. Momand, S. U. Jan, and N. Ramzan, “A systematic and comprehensive survey of recent advances in intrusion detection systems using machine learning: Deep learning, datasets, and attack taxonomy,” *Journal of Sensors*, vol. 2023, Art. no. 6048087, 2023.
- [9] UCI KDD Archive, “KDD Cup 1999 Data.” Accessed: Sep. 29, 2026. [Online]. Available: <https://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>
- [10] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symp. Comput. Intell. Secur. Def. Appl. (CISDA)*, 2009, pp. 1–6.
- [11] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [12] H. Debar, M. Dacier, and A. Wespi, “A revised taxonomy for intrusion-detection systems,” *Annales des Télécommunications*, vol. 55, no. 7–8, pp. 361–378, 2000.
- [13] A. Broder and M. Mitzenmacher, “Network applications of Bloom filters: A survey,” *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [14] S. Dharmapurikar, P. Krishnamurthy, T. S. Sproull, and J. W. Lockwood, “Deep packet inspection using parallel Bloom filters,” *IEEE Micro*, vol. 24, no. 1, pp. 52–61, 2004.
- [15] A. Kirsch and M. Mitzenmacher, “Less hashing, same performance: Building a better Bloom filter,” *Random Structures & Algorithms*, vol. 33, no. 2, pp. 187–218, 2008.
- [16] P. C. Dillinger and P. Manolios, “Bloom filters in probabilistic verification,” in *FMCAD 2004*, ser. LNCS, vol. 3312. Springer, 2004, pp. 367–381.
- [17] S. Mukkamala, G. Janoski, and A. Sung, “Intrusion detection using neural networks and support vector machines,” in *Proc. IEEE IJCNN*, 2002, pp. 1702–1707.
- [18] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, “A deep learning approach to network intrusion detection,” *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 2, no. 1, pp. 41–50, 2018.
- [19] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE S&P*, 2010, pp. 305–316.
- [20] K. He, D. D. Kim, and M. R. Asghar, “Adversarial machine learning for network intrusion detection systems: A comprehensive survey,” *IEEE Commun. Surv. Tutor.*, vol. 25, no. 1, pp. 538–566, 2023.
- [21] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [22] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [23] G. Engelen, V. Rimmer, and W. Joosen, “Troubleshooting an intrusion detection dataset: the CICIDS2017 case study,” in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.